Universidade de Aveiro

Departamento de Eletrónica, Telecomunicações e Informática

API UA

Technical Report

PECI

Equipa

André Reis: 119814

Vasco Oliveira: 119113

Pedro Ferreira: 119240

Carlos Ramos: 119873

Dinis Malhado: 120420

Simão Pereira: 120459

Orientador: Diogo Gomes

June 5, 2026

Abstract

For over a decade, the api.web.ua.pt platform served as a vital digital backbone for the University of Aveiro, centralizing essential services ranging from canteen menus to academic ticketing. However, as the digital landscape evolved, the original infrastructure began to show its age, struggling with outdated security protocols, poor readability, and a lack of modern architectural standards. These mounting technical limitations eventually led to the decommissioning of the legacy system, sparking the urgent need for a complete, from-the-ground-up rebuild to better serve the modern student body. This project outlines the comprehensive modernization of the university's API ecosystem. The reconstruction began by revitalizing core endpoints—including canteen services (sas), RSS-to-JSON translation tools (Rss2Json), institutional data (ArcGIS), real-time parking availability (parques), and academic services (sac). By employing current programming languages and libraries, the fundamental bugs and inefficiencies of the previous system were addressed. To safeguard these assets against external threats, the new APIs were integrated into a robust WSO2 management layer, unified under Keycloak as the platform's identity provider. The result is a secure, high-performance platform complemented by a dedicated documentation portal and an interactive testing framework, ensuring that the university's digital tools are as reliable and accessible as the services they represent.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

The University of Aveiro has long relied on a centralised web services platform — hosted at `api.web.ua.pt` — to expose institutional data to students, faculty, and third-party developers. Through this platform, the university made available a range of services that became embedded in the daily life of its academic community: real-time canteen menus, academic services queue tickets, campus building information, and parking availability, among others.

Despite its relevance, the platform was built over a decade ago on a PHP stack that has since reached end-of-life. Over time, the absence of security updates, the accumulation of unresolved bugs, and the growing incompatibility with modern development practices rendered the system increasingly difficult to maintain. These factors ultimately led to the decommissioning of the legacy platform, leaving a gap in the university's digital infrastructure that directly affected students and developers who depended on it.

1.2 Problem Statement

The core issues that led to the platform's decommissioning can be grouped into three categories.

The first is **security**. The legacy system ran on an outdated version of PHP with known CVEs that had gone unpatched, exposing the platform to exploitation. With no authentication or authorisation layer in place, any consumer could call any endpoint without restriction, making it impossible to protect sensitive data or audit access.

The second is **reliability and performance**. The architecture was not designed to handle concurrent load gracefully. Under peak usage — such as at the start of the academic year or during lunch hours — the server would become unresponsive, degrading the experience for all users simultaneously.

The third is **maintainability**. The codebase lacked modularity and documentation, making it difficult to onboard new developers or extend the platform with additional services. Adding a new API required modifying a tightly coupled codebase with no clear separation of concerns.

1.3 Project Scope Shift

The original plan for this project was to integrate directly with the infrastructure managed by STIC (Serviços de Tecnologias de Informação e Comunicação), the university's IT department. The intended approach was to fully replace the legacy `services.web.ua.pt` backend with the new FastAPI implementation, position WSO2 in front of it using STIC's own API Manager instance, and federate authentication with the university's existing identity provider (`idp.ua.pt`). The broader ambition was to gradually centralise all of the university's web services under a single, governed API platform — one that could become the definitive integration layer for present and future UA systems.

However, at a certain point in the project, communication with STIC ceased. Requests for access to the necessary infrastructure, configuration details, and coordination on deployment went unanswered, making it impossible to proceed with the original integration plan within the project's timeline.

Following consultation with the project supervisor, Diogo Gomes, the decision was made to reorient the project towards a fully self-contained local deployment. Rather than depending on external infrastructure, the team set up its own instances of WSO2 API Manager, Keycloak (in place of `idp.ua.pt`), and a monitoring stack based on Grafana and Loki. This shift in scope did not diminish the project's ambitions — instead, it reframed them: the goal became to *demonstrate the value* that such a platform would bring to the university, delivering a production-equivalent system that is fully documented, containerised, and ready to be migrated to real institutional infrastructure whenever that collaboration becomes possible.

1.4 Goals and Objectives

This project aims to deliver a complete, production-ready replacement for the legacy platform. The main objectives are:

- Migrate all existing services to a modern Python stack using FastAPI, preserving the original endpoints to ensure backward compatibility with existing consumers;
- Introduce a WSO2 API Gateway layer to centralise authentication, authorisation, monitoring, and rate limiting across all services;
- Develop a documentation and interactive testing portal — the *Academic Playground & Innovation* — that makes it straightforward for students and developers to discover, understand, and integrate the available APIs;
- Design the architecture to be modular and extensible, so that new services can be added with minimal effort and no disruption to existing ones.
- Introduce mock APIs to demonstrate the ease of integration for future services and what could have been done if they were to be implemented with real data.

1.5 Document Structure

This report is organised into five chapters. Chapter 2 surveys existing solutions and API gateway platforms, and presents the tools and technologies selected for the project, justifying each choice. Chapter 3 covers the requirements elicitation process and provides a high-level description of the system architecture. Chapter 4 details the implementation of each architectural component, including configuration, local development environment, and integration between services. Chapter 5 outlines the expected outcomes and evaluates the results achieved against the initial objectives.

Chapter 2

Related Work, Tools and Technologies

2.1 Related Work

Two previous projects developed at the University of Aveiro are particularly relevant to this work: the *OPEN DATA UA* master thesis and the final report of the *Academic Playground & Innovation (API)* project. Together, they provide both the conceptual and practical foundations on which this project is built.

The *OPEN DATA UA* thesis surveys existing institutional information sources and web services at UA and proposes a service-oriented architecture based on REST and SOAP for exposing institutional data (e.g., canteen menus, academic information) in a standardized way. It also implements a web portal (APIn) that aggregates services from UA, PT Inovação and SAPO, and introduces an OAuth-based authorization server to protect access to private or sensitive web services. This work is mainly focused on identifying data sources, creating individual web services, and demonstrating how open data and secure access control can be applied within the university context.

The *Academic Playground & Innovation* final report documents the concrete implementation of that vision, namely the api.web.ua.pt portal and the supporting services running on services.web.ua.pt and identity.ua.pt. It describes in detail the API portal, the catalogue of services (UA, PT SAPO, PT Inovação), the documentation and wiki mechanisms, the statistics and administration interfaces, as well as the deployment of an OAuth 1.0a server integrated with the UA Identity Provider using SimpleSAMLphp. This report provides an operational view of how service publication, authentication, authorization and consumption were handled in practice in the first generation of the institutional API platform.

This project can be seen as a next step in this evolution. While *OPEN DATA UA* and the API portal focused on exposing individual services and providing a web-based aggregation and documentation layer, they are tightly coupled to specific technologies and infrastructures that have since become obsolete. In contrast, this project aims

at designing a modern institutional API Gateway and governance model, aligned with current API management practices. Building on the lessons learned from these previous projects, the goal is to provide a more sustainable, secure and governable integration layer for present and future UA systems.

2.1.1 Commercial API Management Platforms

Several commercial and open-source API management platforms address problems similar to those this project aims to solve. **Kong Gateway** is a widely adopted open-source API gateway, offering plugins for authentication, rate limiting, logging, and traffic routing. It is highly performant and extensible, but requires significant infrastructure investment and operational expertise to deploy and maintain at scale. Given that one of the core goals of this project is long-term sustainability and ease of handover to future development teams, introducing a platform with such operational complexity would be counterproductive.

AWS API Gateway and **Azure API Management** are fully managed cloud offerings that eliminate the operational burden of running gateway infrastructure. Both provide built-in support for OpenAPI specifications, OAuth2 integration, and developer portals. However, as hosted proprietary services, they introduce vendor lock-in and are unsuitable for an institutional deployment that must remain under the university's control and on-premises.

RapidAPI represents a different model, functioning primarily as a marketplace and developer portal for discovering and testing third-party APIs. Its interactive testing interface — which allows developers to issue live requests directly from the documentation page — was a direct source of inspiration for the *Academic Playground & Innovation* portal developed in this project.

2.1.2 Comparable Institutional Platforms

Beyond commercial offerings, several universities and public institutions have developed API platforms to expose institutional data to their communities. MIT's *Atlas* and Stanford's developer portals follow a similar model: centralised access to institutional data through authenticated APIs, with developer documentation and sandbox environments.

The common thread across these platforms is the separation between the *gateway layer* — responsible for security, routing, and governance — and the *backend services* that implement the actual business logic. This separation is a key architectural principle adopted in this project, and one that was notably absent from the legacy system, where security concerns, business logic, and data access were all handled within the same tightly coupled PHP codebase with no clear boundaries between layers.

2.2 Tools and Technologies

2.2.1 Python and FastAPI

Python was selected as the primary implementation language for the backend services. Its extensive ecosystem, readability, and widespread adoption in both academia and industry make it a natural fit for a platform that must be maintained by successive generations of students. FastAPI is a modern Python web framework designed for building APIs with high performance and minimal boilerplate. It leverages Python type annotations to generate OpenAPI-compliant documentation automatically at runtime, eliminating the need to maintain separate API specification files. This tight integration between code and documentation is particularly valuable in a context where the OpenAPI specification is consumed directly by both the WSO2 gateway and the developer portal.

2.2.2 WSO2 API Manager

WSO2 API Manager is an open-source, on-premises API management platform that provides a complete solution for publishing, securing, monitoring, and governing APIs. It was selected for this project because it supports the full OAuth2 authorization code flow natively, integrates with external identity providers via OpenID Connect — enabling seamless federation with the Keycloak instance — and exposes a standards-based management interface that can ingest OpenAPI specifications directly. Its open-source nature and on-premises deployment model ensure that the university retains full control over the platform without dependency on third-party cloud providers.

2.2.3 Keycloak

Authentication and authorisation are handled through the OAuth2 protocol, delegated to a **Keycloak** instance configured with a dedicated `ua` realm. Keycloak was chosen for its mature support for OAuth2 and OpenID Connect, its flexible role-based access control, and its ability to federate with external identity providers. The WSO2 gateway validates tokens issued by Keycloak before forwarding requests to the backend, and access scopes are mapped to user roles — `admin`, `professor`, and `aluno` — allowing fine-grained control over which endpoints each profile can consume. The Docusaurus developer portal is also integrated with Keycloak, enabling users to authenticate directly from the documentation interface.

2.2.4 Docusaurus and Stoplight Elements

The developer portal is built with **Docusaurus**, a React-based static site generator maintained by Meta. It was chosen for its native support for Markdown and MDX content, built-in versioning, and low maintenance overhead — documentation updates require only editing text files, with no application code changes. Interactive API testing is provided by **Stoplight Elements**, an open-source React component that renders a full API explorer from an OpenAPI specification. All test requests issued through the portal are routed through the WSO2 gateway, ensuring that the developer experience accurately reflects production behaviour.

2.2.5 Memcached

Memcached is a high-performance, in-memory key-value store used in this project to cache upstream API responses. Given that certain data sources — such as the UA WSO2 canteen menu endpoint — are slow to respond and return data that changes at most once per day, caching responses eliminates redundant upstream calls and significantly reduces response times for end consumers. Memcached was preferred over alternatives such as Redis for its simplicity and minimal resource footprint, which is particularly relevant given the constrained hardware available for deployment.

2.2.6 Grafana, Loki and Promtail

Platform observability is provided by **Grafana** and **Loki**. Loki collects and indexes logs produced by the WSO2 API Manager, while Grafana provides a dashboard layer for visualising API usage metrics, error rates, and traffic patterns per endpoint. Log ingestion is handled by **Promtail**, which ships WSO2 logs to Loki in real time. A pre-provisioned dashboard is included in the repository, giving operators an immediate view of platform health without manual configuration.

2.2.7 Docker

All platform components are containerised using **Docker** and orchestrated via **Docker Compose**. The services run on a private Docker network (`gateway-net`), isolating them from external access. WSO2 API Manager is the only component exposed externally, acting as the single entry point for all API traffic.

Chapter 3

Requirements and Architecture Overview

3.1 Requirements Elicitation

The requirements for this project were gathered through direct engagement with the primary stakeholders. Meetings with STIC (Serviços de Tecnologias de Informação e Comunicação) and the project supervisor, Diogo Gomes, provided the functional and non-functional constraints that shaped the architecture.

3.1.1 Functional Requirements

- **FR1 – Service Centralisation and Exposure:** A WSO2 layer must exist that acts as a centralised API Gateway to expose the University of Aveiro's services uniformly.
- **FR2 – Identity and Permissions:** The system must implement authentication and authorisation mechanisms, ensuring access control per API according to each user's profile.
- **FR3 – Control and Monitoring:** The platform must allow detailed control and monitoring of all service accesses, including logging and traceability of calls made.
- **FR4 – Documentation Availability:** Documentation must be available for the use of each API, including description of endpoints, parameters, authentication, and usage examples.
- **FR5 – Continuity and Compatibility:** The platform must maintain the same endpoints, ensuring compatibility with already integrated systems and avoiding service interruptions.

3.1.2 Non-Functional Requirements

- **NFR1 – Security:** The system must comply with security best practices, mitigating vulnerabilities identified in the previous solution.
- **NFR2 – Performance:** API requests must be processed in less than 3 seconds under normal operating conditions.
- **NFR3 – Technical Scalability:** The API Gateway must support increased load and growth in the number of consumers without significant service degradation.
- **NFR4 – Functional Scalability:** The introduction and integration of new APIs must be simple, quick, and have minimal impact on existing services.
- **NFR5 – Maintainability:** The system must adopt software development best practices and keep code readable, modular, and well-documented.
- **NFR6 – Interface Usability:** The documentation portal must present a simple, clear, and intuitive interface, promoting adoption of the available services.

3.2 System Architecture

The platform is organised into five interconnected layers, as illustrated in Figure 3.1. All components run on a private Docker network (`gateway-net`), with WSO2 as the sole externally exposed entry point. The following subsections describe each layer and how they interact.

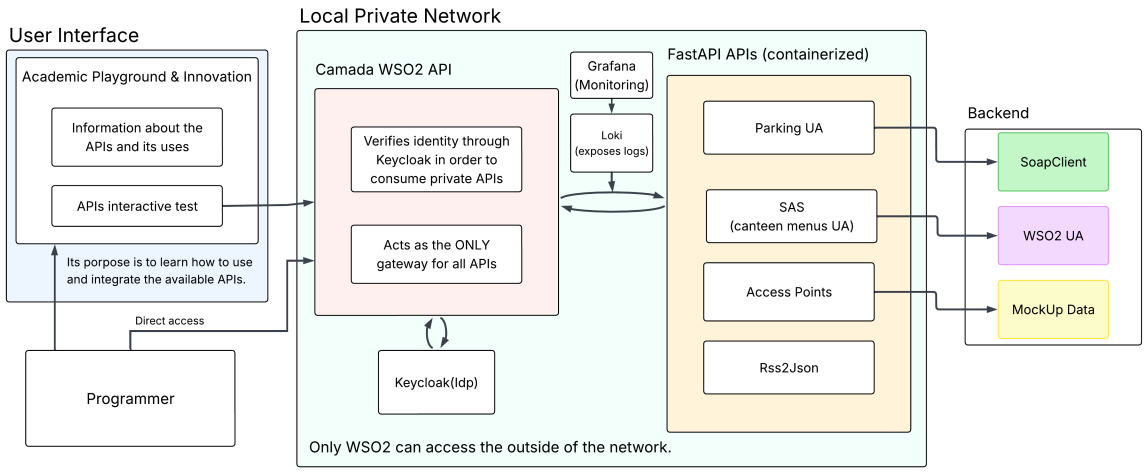

Figure 3.1: System Architecture Overview

3.2.1 Consumer-Facing Layer

The entry point for human users is the *Academic Playground & Innovation* portal, a static documentation website that exposes the full API catalogue alongside an interactive testing interface. When a user wishes to test an endpoint, the portal issues the request not directly to the backend, but to the WSO2 gateway — the same path taken by any external application. Authentication from the portal is handled by redirecting the user to Keycloak, obtaining a token, and attaching it automatically to subsequent test requests.

3.2.2 Identity and Access Control

Keycloak sits at the centre of the platform's security model. It is the sole authority on user identity: all login flows, regardless of their origin (portal, Developer Portal, or external client), are redirected to Keycloak. Upon successful authentication, Keycloak issues a signed JWT token that encodes the user's assigned roles. This token is then presented to WSO2, which validates it and uses the role information to determine what the user is permitted to access. No credentials are handled by any other component.

3.2.3 API Gateway

WSO2 API Manager is the single externally exposed component of the platform. Every API request — whether originating from the documentation portal, a developer's application, or a third-party consumer — must pass through it. WSO2 validates the Bearer token provided by the caller, checks that the caller holds an active subscription for the requested API, applies rate limiting, and only then forwards the request to the appropriate backend service. Responses follow the reverse path back to the caller. WSO2 also exposes a Developer Portal where consumers can discover APIs, create applications, and generate access tokens.

3.2.4 Backend Services

The business logic of each individual service is implemented as an independent FastAPI module. These services are not directly reachable from outside the Docker network; they only receive traffic forwarded by the WSO2 gateway. Depending on the service, data is retrieved from different upstream sources: some services query the university's own WSO2 instance for live institutional data (such as canteen menus), others interface with legacy SOAP systems, and one stores its state locally in a CSV file. Frequently accessed upstream responses are cached in Memcached to reduce latency and relieve pressure on external data sources.

3.2.5 Observability

All components within the platform emit logs that are collected by Promtail and forwarded to Loki for centralised storage and indexing. Grafana connects to Loki as a datasource and presents the aggregated data through pre-provisioned dashboards, giving operators a real-time view of gateway traffic, error rates, and API usage patterns. This layer does not sit in the request path and has no effect on API response times.

Chapter 4

Implementation

4.1 Local Development Environment

Given the constraints described in Section 1.3, the entire platform was built and validated in a self-contained local environment. All components are orchestrated by a single `docker-compose.yml` file, allowing the full stack — gateway, identity provider, backend APIs, and monitoring — to be brought up with a single command on any machine with Docker installed.

4.1.1 Network Isolation

All services communicate over a shared Docker network named `gateway-net`, declared as external so that it can be created once and reused across multiple Compose projects. Within this network, containers resolve each other by service name (e.g. the SAS service reaches Memcached at the hostname `memcached`, and Promtail reaches Loki at `loki`). No backend service binds a port to the host machine; only WSO2 (ports 8243, 8280, 9443), Keycloak (8080), Loki (3100), and Grafana (3000) are reachable from outside the network. This mirrors the intended production topology, where the gateway is the sole external entry point.

4.1.2 Persistent State via Volumes

Three named volumes are defined to preserve state across container restarts:

- `wso2_logs` — shared between the WSO2 container and Promtail, allowing the log agent to read WSO2's `wso2carbon.log` without any direct coupling between the two containers;
- `wso2_db` — persists WSO2's internal H2 database, retaining published APIs, applications, and subscriptions between restarts;

- `keycloak_data` — persists the Keycloak database, so that the `ua` realm, users, and client configuration survive container recreation.

4.1.3 Service Composition

Table 4.1 summarises the services defined in the Compose file and their roles within the platform.

Table 4.1: Docker Compose services

Service	Role
`wso2-apim`	API Gateway and Developer Portal. Depends on all backend services being available before starting.
`keycloak`	Identity Provider. Imports the `ua` realm automatically on first start via `-import-realm`.
`loki`	Log aggregator. Receives log streams from Promtail.
`promtail`	Log agent. Reads the shared `wso2_logs` volume and forwards entries to Loki.
`grafana`	Observability dashboard. Datasources and dashboards are provisioned automatically from mounted configuration files.
`sas`	Canteen menus API. Depends on Memcached for response caching.
`parques`	Parking availability API.
`rss2json`	RSS to JSON converter API.
`ap`	Access Points mock API.
`memcached`	In-memory cache, used exclusively by the SAS service.

4.1.4 Reproducibility

A key design principle of this environment is that no manual configuration step should be required after `docker compose up`. WSO2 loads its configuration from a mounted `deployment.toml`; Keycloak imports the realm from a versioned JSON file; Grafana provisions its datasource and dashboards from YAML and JSON files mounted into the container. The WSO2 API catalogue is restored automatically at startup via `import-all.sh`. The result is a fully deterministic environment that can be reproduced identically on any development machine.

4.1.5 Local Environment vs. Production Deployment

While the local environment faithfully replicates the platform's intended behaviour, several differences would need to be addressed in a real institutional deployment.

Table 4.2: Local environment vs. production deployment

Aspect	**Local**	**Production**
Orchestration	Docker Compose on a single host	Kubernetes cluster (UA infrastructure)
Identity	Self-hosted Keycloak	Federated with `idp.ua.pt`
TLS/HTTPS	Not enforced locally	Required on all external endpoints
WSO2 database	Embedded H2	External MySQL or PostgreSQL
Scaling	Single instance per service	Horizontal pod autoscaling
Secrets	Plain-text `.env` files	Kubernetes Secrets or Vault
Log retention	Ephemeral volume	Persistent storage with retention policy

Despite these differences, the containerised approach ensures that migrating to a production environment is primarily an infrastructural exercise rather than a code change. Each service already runs in an isolated container with clearly defined dependencies, environment variables, and network boundaries — the same primitives that a Kubernetes deployment would use.

4.2 WSO2 API Manager

WSO2 API Manager acts simultaneously as the API Gateway and as a Service Provider within the platform's security architecture. It is the sole component exposed to the outside world, intercepting every inbound request before it reaches any backend service. Rather than managing user identities itself, WSO2 delegates all authentication to Keycloak, focusing exclusively on validating tokens and enforcing subscription-based access control over the published APIs.

4.2.1 Configuration

The core WSO2 configuration is defined in `deployment.toml`. The most significant settings establish the federation with Keycloak: WSO2 is pointed at Keycloak's OIDC endpoints (Authorization, Token, and Userinfo), and a confidential client registration is referenced so that the token exchange between the two systems can be performed securely. CORS headers are also configured at this level to permit cross-origin requests originating from the documentation portal, which is necessary for the interactive testing feature to function correctly from a browser context.

4.2.2 API Registration and Lifecycle

APIs are defined and published through the WSO2 API Publisher. Each API (SAS, Parques, Rss2Json, and the Access Points mock API) is created with its backend context, endpoint URL, and resource definitions. Once published, the API becomes discoverable and subscribable in the Developer Portal.

To ensure the deployment is fully reproducible, all API definitions are exported as ZIP files using `apictl`, the WSO2 command-line tool, and stored under version control. An import script (`import-all.sh`) re-registers all APIs automatically on startup, meaning the entire API catalogue is restored without any manual intervention when the stack is brought up from scratch. This was mainly done because the project was kept on a repository and it was the most convenient way to share it between team members, in a real scenario, there would be no need to import the APIs as all the configurations would be stored in the machine's docker volumes.

4.2.3 Access Control and Subscriptions

Access to every published API is governed by a subscription model. A consumer must create an Application in the Developer Portal and subscribe that Application to the desired API. Upon subscription, a set of OAuth2 Bearer tokens (Production Keys) is generated for that Application. Every subsequent API request must carry one of these tokens in the `Authorization` header.

When a request arrives at the gateway, WSO2 performs the following checks before forwarding it to the backend:

- validates the token signature and expiry;
- verifies that the Application owning the token holds an active subscription for the requested API path and version;
- applies any configured rate limiting policies.

Only if all checks pass does the internal Synapse engine forward the request to the corresponding FastAPI backend service and relay the response back to the client.

4.2.4 Role Mapping — Translating Keycloak Roles into WSO2 Privileges

Although WSO2 delegates authentication to Keycloak, authorisation within the WSO2 portals (i.e. what a user is permitted to do in the Publisher and Developer Portal) depends on WSO2-internal roles. Keycloak and WSO2 use different role nomenclatures: Keycloak may define a role such as `api_consumer`, while WSO2 requires an internal role of `Internal/subscriber` to allow a user to subscribe to APIs in the Developer Portal.

To bridge this semantic gap, a Role Mapping table was configured in the Identity Provider settings of the WSO2 Carbon admin panel. The mapping operates as follows:

1. **Claim extraction:** when the user authenticates, the Keycloak token or Userinfo response includes a roles claim — an array of role names assigned to that user in the Keycloak `ua` realm.

2. **Translation:** WSO2 reads those IdP roles and maps each one to one or more local roles using the configured dictionary. For example, `api_consumer` is translated to `Internal/subscriber`, granting the user the ability to discover and subscribe to APIs in the Developer Portal.

3. **Injection via JIT Provisioning:** the translated roles are assigned to the user's profile at the moment it is silently created in WSO2's PRIMARY user store (see Section 4.3.4).

Without this translation layer, federation would be authentication-only: users would be able to log in via Keycloak, but would encounter empty screens or `403 Forbidden` responses throughout the WSO2 portals. The role mapping closes that gap and ensures that access privileges defined in Keycloak propagate automatically into WSO2.

4.3 Keycloak

Keycloak serves as the central Identity Provider (IdP) for the platform, handling all user authentication through the OpenID Connect (OIDC) protocol. No user passwords are stored or validated by WSO2 or by any of the backend API services; all authentication flows are directed to Keycloak.

4.3.1 Realm Configuration

A dedicated realm named `ua` was created to isolate the platform's identity configuration from any default Keycloak settings. The realm defines the token expiry policies, OIDC endpoints, and the set of clients and roles used across the platform.

4.3.2 Client Setup and CORS

A confidential OIDC client with the ID `wso2` was registered within the `ua` realm to represent the WSO2 API Manager. The client is configured with:

- **Client Authentication enabled** (confidential client): ensures that only the WSO2 server, which holds the client secret, can exchange authorisation codes for access tokens;
- **Standard Flow (Authorization Code Flow) enabled**: the user is redirected to Keycloak's login page, authenticates, and is returned to the WSO2 Developer Portal with a short-lived authorisation code, which WSO2 then exchanges for a JWT access token;
- **Web Origins and Redirect URIs**: configured to permit requests from the WSO2 portal URLs, preventing browsers from blocking the OIDC redirect flow with Cross-Origin Resource Sharing (CORS) errors.

4.3.3 User Roles

Three roles are defined in the `ua` realm, corresponding to the expected consumer profiles of the platform:

- `admin` — full access to both the API Publisher and Developer Portal;
- `professor` — access to a defined subset of APIs;
- `aluno` — access to public and student-facing APIs.

These roles are included in the tokens issued by Keycloak and mapped to WSO2 internal roles as described in the previous section.

4.3.4 Just-in-Time Provisioning

WSO2 maintains its own internal user store (the `PRIMARY` domain) to track Applications and API subscriptions. For a user authenticated via Keycloak to be able to create Applications and subscribe to APIs in the Developer Portal, a corresponding record must exist in this store.

Just-in-Time (JIT) Provisioning was configured on the WSO2 side to handle this automatically. When a user logs into the Developer Portal via Keycloak for the first time, WSO2 silently creates a profile for that user in the `PRIMARY` store, assigns the translated local roles, and completes the login — all without any visible interruption to the user. On subsequent logins, the existing profile is updated if the user's roles in Keycloak have changed.

4.3.5 The Complete Authentication and API Call Flow

To illustrate how all components interact, the following describes the end-to-end flow from a user's first login to a successful API call:

1. The user navigates to the WSO2 Developer Portal and clicks *Login*.
2. WSO2 redirects the browser to Keycloak's authorisation endpoint.
3. The user authenticates against Keycloak (username and password).
4. Keycloak issues an authorisation code and redirects the browser back to WSO2.
5. WSO2 exchanges the authorisation code for a JWT access token at Keycloak's token endpoint, using its client secret.
6. WSO2 reads the user's roles from the token, applies the Role Mapping, and creates (or updates) the user's profile in the `PRIMARY` store via JIT Provisioning.
7. The user is now logged into the Developer Portal. They create an Application, subscribe it to an API (e.g. *Parques 1.0.0*), and generate Production Keys (a Bearer token).
8. To call the API, the user sends an HTTP request to the WSO2 gateway with the header `Authorization: Bearer <token>`.
9. The WSO2 gateway intercepts the request, validates the token, and verifies that the Application has an active subscription for the requested API path.
10. On success, the request is fowarded to the corresponding FastAPI backend service and returns the response to the client.

4.4 APIs — ServicesUAPT

The API UA platform was implemented in Python using the FastAPI framework, following a modular router-based architecture. Each service is encapsulated in its own router module, registered in the main application entry point, and exposed under a

dedicated URL prefix. This design ensures a clear separation of concerns between services and facilitates the integration of new APIs in the future without modifying existing ones.

The main application initialises the FastAPI instance, registers all routers, and manages the lifecycle of shared resources — such as the SOAP client used by the Parking Lots API — through FastAPI's lifespan context manager, ensuring proper initialisation and cleanup on startup and shutdown respectively.

For services that consume external data sources, a caching layer was introduced to reduce upstream load and improve response times. The Canteen Menus API uses **Memcached** as its caching backend, storing the full week's menu data in memory and refreshing it daily at midnight. The RSS to JSON API uses a file-based cache, storing each converted feed as a local file identified by a SHA-1 hash of its URL, with background refresh tasks keeping the cache up to date without blocking client responses.

All APIs maintain backward compatibility with the legacy PHP services they replace, preserving the same endpoint structure, query parameters, and response formats — including XML, JSON, and JSONP — ensuring a transparent migration for existing clients.

4.4.1 SAS — Canteen Menus

The Canteen Menus API provides access to the University of Aveiro's canteen menu information, exposing the weekly and daily meal schedules for the various canteens across the university campus. The API acts as a compatibility layer over the University of Aveiro's internal WSO2 API, maintaining the same response format as the legacy service to ensure backward compatibility with existing clients.

The API fetches menu data from the UA WSO2 gateway endpoint `mysas_mysas/v1/Refeicoes/GetAgendaMenusEntreDatas`, which returns the full week's menu schedule. This data is then filtered in memory according to the requested place and date parameters, avoiding repeated calls to the upstream service for different parameter combinations.

The canteens covered by this API are Santiago, ESTGA, ESAN, and Restaurante Universitário. Each menu entry exposes the following attributes:

- **Canteen** — the full name of the canteen;

- **Meal** — the meal period (e.g. Almoço);
- **Date** — the date of the menu in ISO 8601 format (`YYYY-MM-DD`);
- **Items** — the list of meal components available for that menu.

Caching Strategy

A key feature of the SAS API is its caching mechanism, implemented using Memcached. Rather than fetching data from the upstream WSO2 API on every request, the API stores the full week's menu data in Memcached under a single cache key (`ementas:all:week`). This cached data is then filtered in memory for each request, making any combination of place and date parameters fast to serve without additional upstream calls.

The cache expiry is calculated dynamically as the number of seconds remaining until midnight, ensuring the cache is refreshed daily with up-to-date menu information. Additionally, the API supports HTTP caching through `ETag` and `Cache-Control` headers — if the client already holds the current version of the data, the API returns a `304 Not Modified` response without a body, reducing unnecessary data transfer.

SAS API Implementation Stack

The SAS API was implemented in Python using the FastAPI framework. The application architecture relies on several key technologies:

- **FastAPI** — the web framework used to define and expose the REST endpoint, providing automatic OpenAPI documentation generation and request validation;
- **Uvicorn** — the ASGI server responsible for running the application and exposing the API through HTTP requests;
- **Memcached** — an in-memory caching system used to store the full week's menu data, reducing the number of upstream requests and improving response times;
- **pymemcache** — the Python client library used to communicate with the Memcached server, providing get, set, and flush operations through a thin wrapper centralising all cache logic;
- **requests** — used to perform HTTP requests to the upstream UA WSO2 API to fetch the menu data;
- **xmltodict** and **dicttoxml** — used to handle XML serialization of the response data for the default XML format;

- **Pydantic** — used for request parameter validation and configuration management.

The API exposes the following endpoint:

- `GET /sas/ementas` — returns canteen menus, accepting the following query parameters:
  - **place** — the canteen to filter by; accepted values are `santiago`, `estga`, `esan`, `rest`, and `all` (default: `all`);
  - **date** — the time range of menus to return; accepted values are `day` for today's menus only and `week` for the full week (default: `day`);
  - **format** — the response format; accepted values are `xml`, `json`, and `jsonp` (default: `xml`);
  - **cb** — the JavaScript callback function name, required when `format=jsonp`;
  - **help** — when present, redirects the client to the API documentation;
  - **clear_cache** — when present, flushes the Memcached cache and returns a `200 OK` response.

4.4.2 Parques — Parking Availability

The Parking Lots API provides real-time occupancy information for the University of Aveiro's parking facilities. Rather than directly exposing the existing SOAP service, this API acts as a REST/JSON wrapper around the `LotacaoParques` SOAP service, making it easier to consume by modern clients and applications.

The service retrieves live data from the University of Aveiro's internal SOAP endpoint, hosted at `score.ua.pt`, which exposes a single operation — `Lotacao()` — returning the current occupancy status for all parking lots. The API enriches this data with additional static information, such as each parking lot's display name and geographical coordinates (latitude and longitude), which are maintained in a hardcoded mapping structure within the application.

The parking lots covered by this API are distributed across the University of Aveiro campus and affiliated locations, including Residencias, Biblioteca, ZTC, Subterraneo, Ceramica, Linguas, Incubadora, ISCAA Publico, ISCAA Funcionarios, and ESTGA. Each parking lot is identified by a unique identifier (e.g. `P8`, `P1|4`) and exposes the following attributes:

- **Parque** — the internal name of the parking lot as returned by the SOAP service;

- **Display Name** — a human-readable name for the parking lot;
- **Capacidade** — the total parking capacity;
- **Ocupado** — the number of currently occupied spaces;
- **Livre** — the number of currently available spaces;
- **Latitude and Longitude** — the geographical coordinates of the parking lot.

Parking Lots API Implementation Stack

The Parking Lots API was implemented in Python using the FastAPI framework. The application architecture relies on several key technologies:

- **FastAPI** — the web framework used to define and expose the REST endpoints, providing automatic OpenAPI documentation generation and request validation;
- **Uvicorn** — the ASGI server responsible for running the application and exposing the API through HTTP requests;
- **Zeep** — the Python SOAP client library used to communicate with the `Lotacao Parques` SOAP service, wrapped in an asynchronous executor to avoid blocking the FastAPI event loop;
- **Pydantic** — used for data validation and serialization of the API responses, ensuring type safety and consistent output formats.

The API exposes the following endpoints:

- `GET /parques/parques` — returns occupancy data for all parking lots, with an optional `id` parameter accepting one or more comma-separated parking lot identifiers (e.g. `?id=P8,P9`);
- `GET /parques/health` — a liveness probe endpoint that confirms the API and SOAP client are operational;
- `GET /parques/occupancy` — returns occupancy data with an optional filter by parking lot name;
- `GET /parques/occupancy/{parque_name}` — returns occupancy data for a single parking lot identified by name.

4.4.3 Rss2Json

The RSS to JSON API provides a simple interface for converting RSS feeds into JSON format, making it easier for modern clients and applications to consume RSS content without having to parse XML directly. The API accepts any valid RSS feed URL and returns its contents as a structured JSON object, with optional JSONP support for cross-origin requests.

A key feature of this API is its caching mechanism. To avoid unnecessary network requests and reduce latency, the API stores the converted feed data in a local cache directory. Each cache file is identified by a SHA-1 hash of the RSS feed URL, ensuring unique cache entries per feed. When a cached version of a feed exists, the API serves it immediately and schedules a background refresh to keep the cache up to date. When no cache exists, the feed is fetched, converted, and stored before being returned to the client.

This stale-while-revalidate caching strategy ensures that the API always responds quickly, even when the upstream RSS feed is slow or temporarily unavailable, while still keeping the data reasonably fresh.

RSS2JSON API Implementation Stack

The RSS2JSON API was implemented in Python using the FastAPI framework. The application architecture relies on several key technologies:

- **FastAPI** — the web framework used to define and expose the REST endpoint, providing automatic OpenAPI documentation generation and request validation;
- **Uvicorn** — the ASGI server responsible for running the application and exposing the API through HTTP requests;
- **httpx** — an asynchronous HTTP client used to fetch the RSS feed from the provided URL, supporting redirects and configurable timeouts;
- **xmltodict** — used to parse the XML/RSS content into a Python dictionary, which is then serialized into JSON;
- **Pydantic** — used for request parameter validation, ensuring the RSS URL is provided and correctly formatted.

The API exposes the following endpoint:

- `GET /rss2json` — converts a RSS feed to JSON, accepting the following query parameters:

  - **rss** — the URL of the RSS feed to convert (required, must be URL-encoded);
  - **callback** — an optional JavaScript function name; when provided, the response is returned in JSONP format instead of plain JSON, enabling cross-origin requests from browser-based clients.

The caching behaviour follows this logic:

- **Cache hit** — the cached JSON is served immediately and a background task is scheduled to refresh the cache asynchronously, without blocking the response;
- **Cache miss** — the RSS feed is fetched synchronously, converted to JSON, stored in the cache, and returned to the client;
- **Cache storage** — each feed is stored as a file identified by the SHA-1 hash of its URL, in a dedicated cache directory within the application.

4.4.4 Access Points — Mock API

The API UA platform is not merely a system that centralizes the various services offered by the University of Aveiro. Rather, it is designed to facilitate easy integration of new APIs that may be developed in the future. To demonstrate this extensibility, we created a mock API to simulate the integration of new web services into the platform.

Our supervisor recommended developing a mock API for the University of Aveiro's access points, containing useful information such as their geographical locations, maximum connection capacity, and other data relevant to the academic community.

To store the mock API data we created a MySQL database hosted on the IETTA infrastructure. This decision was driven by the fact that, concurrently with this project, we attended a relational databases course that provided access to the IETTA database for class exercises and the course project. Given that existing access and the operational convenience it provided, we obtained permission from the course coordinator to use the IETTA database for persisting the data for our mock API.

The database schema contains two primary tables: one for access points and another for the departments where each access point is located. Each table includes several attributes, which are represented in the entity–relationship diagram (ERD) shown in Figure (4.1). The ERD was designed to organise and document the database structure.

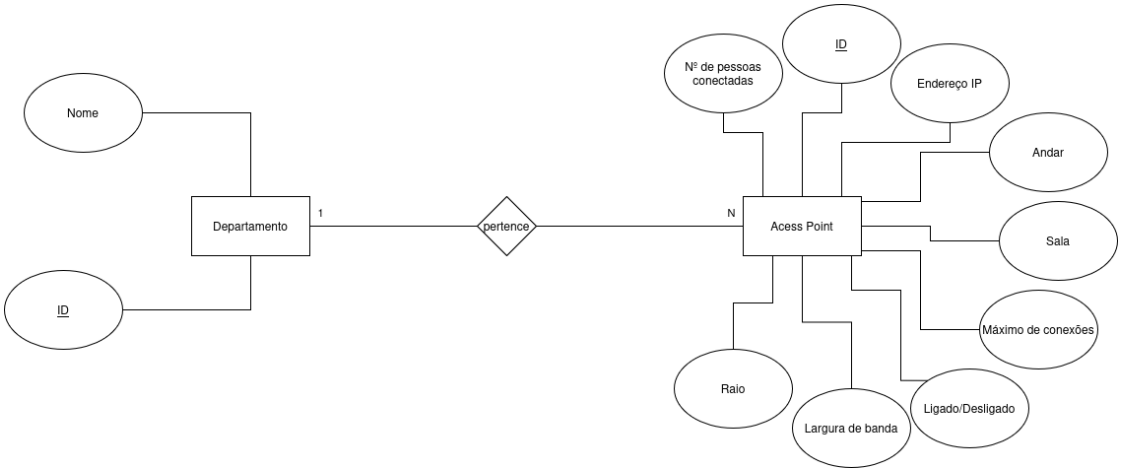

Figure 4.1: ERD for the Mock API Access Points Database

Mock API Implementation Stack

The mock API was implemented in Python, running on a virtual environment to isolate project dependencies and ensure a controlled, reproducible development environment without conflicts with other system installations. The application architecture relies on several key technologies:

- **Uvicorn** — the ASGI server responsible for running the application and exposing the API through HTTP requests;
- **PyODBC** — the library used to establish the connection between the Python application and the SQL Server database, using the ODBC protocol;
- **ODBC** — a standard that enables access to different database management systems in a uniform and language-independent way, providing database abstraction;

The overall architecture diagram is shown in Figure (4.2), illustrating the integration of these components within the broader API UA platform.

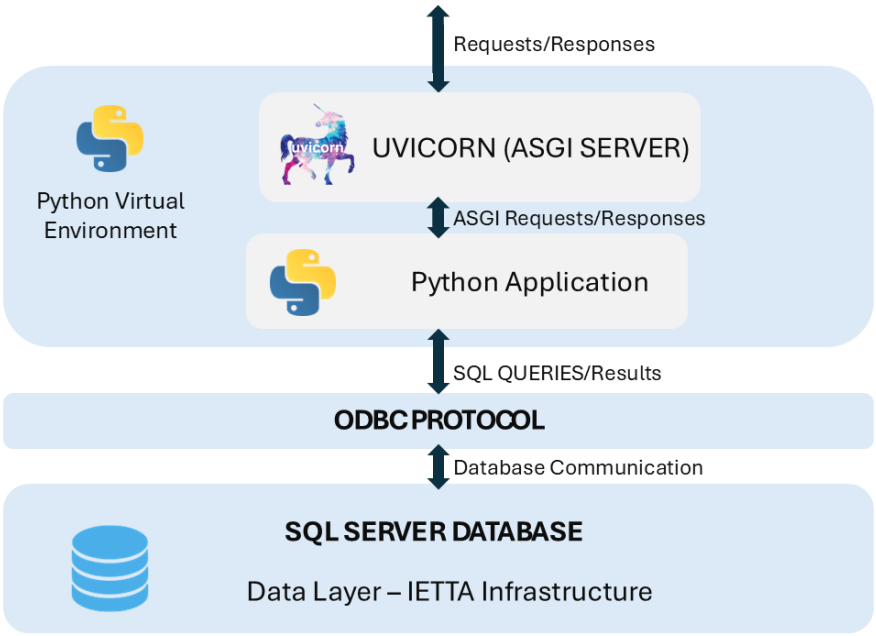

Figure 4.2: Mock API architecture diagram

4.5 Academic Playground & Innovation — Documentation Portal

The user-facing layer is the *Academic Playground & Innovation* portal. It serves two purposes: providing comprehensive documentation on how to use each API, and allowing interactive testing of all available endpoints directly in the browser. The goal is to make it easy for developers to learn how to integrate the APIs available into their own projects. Once they are familiar with the methods, the portal serves as a reliable reference guide.

4.5.1 Structure and Content

The portal was developed using **Docusaurus**, a static site generator built on React and maintained by Meta. This choice is justified by its native support for Markdown and MDX, built-in documentation versioning, and ease of maintenance — updating content requires only editing text files, with no need to touch application code.

This architecture facilitates scaling the number of APIs. Adding a new API to the portal requires no manual frontend development. Administrators only need to declare a new Scalar plugin instance in the configuration file, pointing it to the respective OpenAPI YAML definition:

Listing 4.1: Declarative registration of the Rss2json API using the Scalar plugin

```
1 [
```

```
  '@scalar/docusaurus',
  {
    id: 'rss2json',
    label: 'Rss2json',
    route: '/api/rss2json',
    showNavLink: false,
    configuration: {
      url: `${baseUrl}openAPI-rss2json.yaml`,
      metaData: { title: 'Rss2json' },
      proxyUrl: 'https://proxy-api-orcin.vercel.app/api/proxy',
      ...commonScalarConfig,
    },
  }
]
```

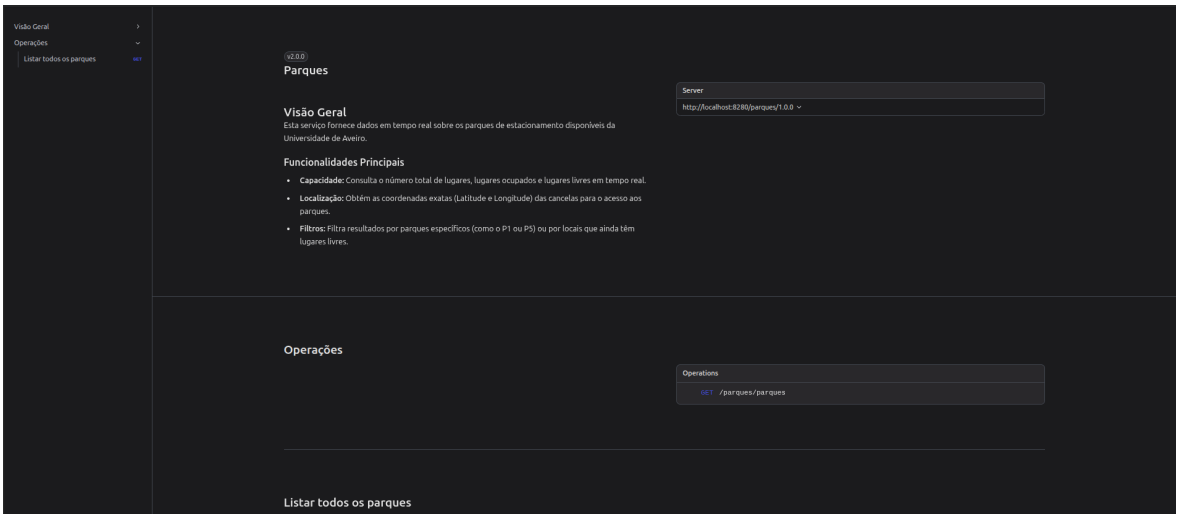

Figure 4.3: Scalar API explorer interface automatically rendering the endpoint catalog from the OpenAPI specification.

4.5.2 Interactive Testing

To provide interactive testing capabilities directly in the browser, was integrated Scalar into the Docusaurus site. Rather than communicating with the FastAPI backend directly, all test requests issued from the portal are routed through the **WSO2 API Gateway**, mimicking the exact path taken by any external consumer. This is accomplished by setting the WSO2 public endpoint as the target server in the OpenAPI specification:

Listing 4.2: OpenAPI server configuration targeting WSO2

```
{
  "servers": [
```

```
    { "url": "https://api.ua.pt/v1" }
  ]
}
```

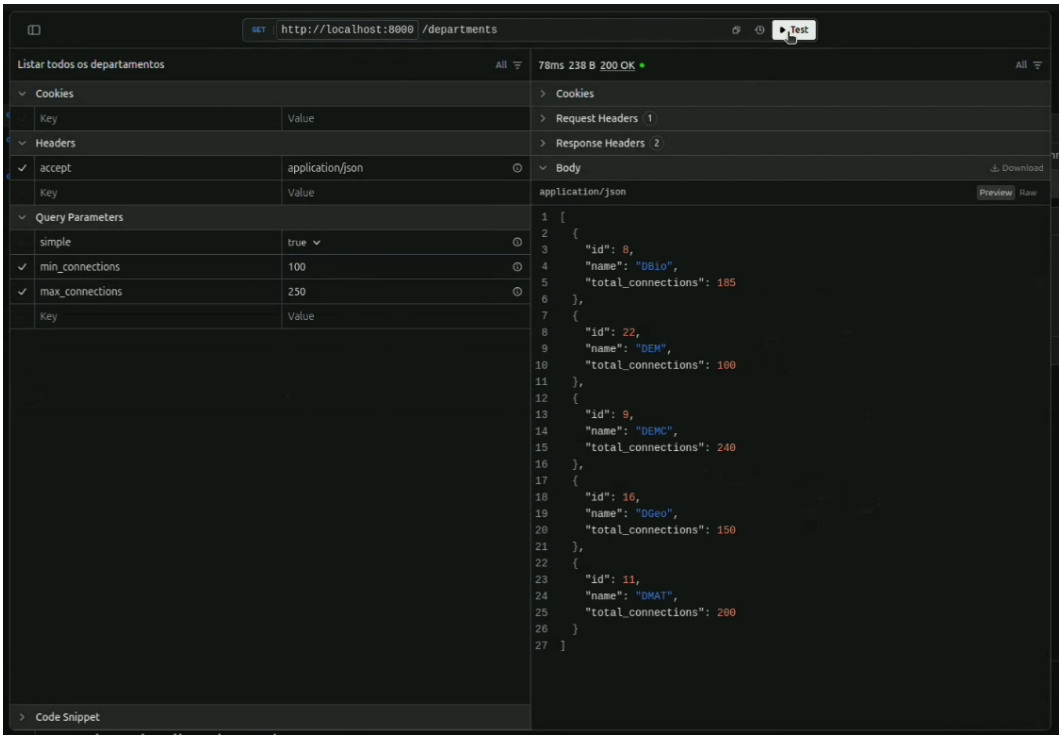

Figure 4.4: Interactive API explorer powered by Scalar, showing a successful request routed through WSO2.

A fundamental challenge when integrating interactive API testing within a browser environment is the Cross-Origin Resource Sharing (CORS) policy. Since the portal interacts with various external APIs that may lack proper CORS configurations, a dedicated middleware proxy was developed using Node.js and Express.

When the Scalar interface attempts to call a restricted API, the request is routed to the proxy. The proxy server performs the actual HTTP request to the target destination, attaching necessary wildcard CORS headers to the response before forwarding the payload back to the Docusaurus frontend.

Listing 4.3: Node.js Express Proxy for CORS circumvention and header injection

```
async function handler(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', '*');

  if (req.method === 'OPTIONS') return res.status(200).end();
```

```
  let targetUrl = req.query.url || req.query.scalar_url;
  if (!targetUrl) return res.status(400).json({ error: "Missing␣
     target␣URL" });

  try {
    const fullUrl = new URL(decodeURIComponent(targetUrl));
    const response = await fetch(fullUrl.toString(), {
      method: req.method,
      headers: {
        'Accept': '*/*',
        'User-Agent': 'Mozilla/5.0␣(Windows␣NT␣10.0;␣Win64;␣x64)
           ␣AppleWebKit/537.36',
        'Connection': 'keep-alive'
      }
    });

    const data = await response.text();
    res.setHeader('Content-Type', 'application/json');
    res.status(response.status).send(data);
  } catch (error) {
    res.status(500).json({ error: 'Gateway␣error', details:
       error.message });
  }
}
```

4.5.3 Keycloak Integration

To handle private APIs and administrative tools, the portal integrates seamlessly with **Keycloak** for Single Sign-On (SSO). Using the `@react-keycloak/web` library, the application maintains a global authentication state across the static site. To ensure compatibility with Docusaurus' Static Site Generation (SSG) process, protected React components are wrapped in a `<BrowserOnly>` boundary, preventing server-side rendering errors.

A dynamic Role-Based Access Control (RBAC) system was implemented directly in the UI. Components such as the Navigation Bar automatically read the JSON Web Token (JWT) provided by Keycloak and filter their visibility based on the user's assigned roles.

Keycloak Role	**Visible APIs**
Public (Unauthenticated)	Canteen Menus, RSS to JSON
User (Standard Access)	Public APIs, Parking Lots
Administrator	All APIs, Access Points API

Table 4.3: Mapping of UI component visibility based on Keycloak RBAC roles.

The custom `ProtectedDropdown` component reads these declarative properties and filters the endpoints accordingly:

Listing 4.4: Role-based access control configuration in Docusaurus navbar

```
{
  type: 'custom-ProtectedDropdown',
  label: 'Servicos',
  position: 'left',
  items: [
    { label: 'Parques', to: '/api/parques' }, // Public
    {
      label: 'Rss2json',
      to: '/api/rss2json',
      customProps: { allowedRoles: ['admin_role', '
         publisher_role'] }
    }
  ]
}
```

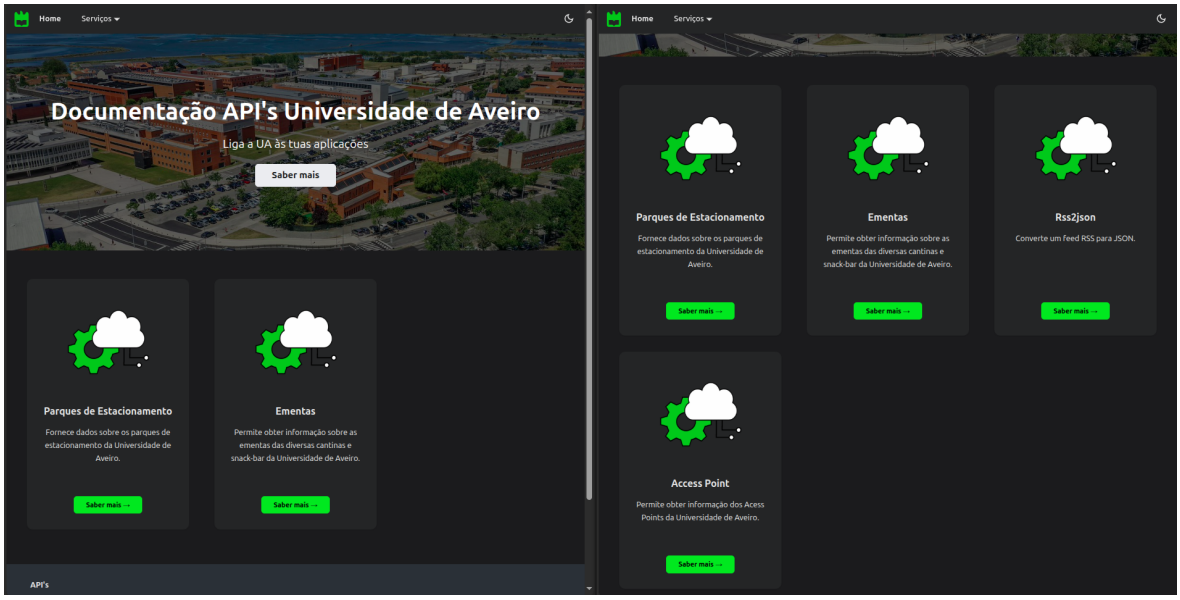

Figure 4.5: Dynamic UI rendering: Public view (left) vs. Administrator view with expanded access (right).

4.6 Monitoring — Grafana, Loki and Promtail

The monitoring stack provides full observability over the platform's runtime behaviour. Access to the Grafana interface is restricted to the platform administrator — the dashboards expose internal gateway metrics, error patterns, and per-API traffic data that are operationally sensitive and not intended for general consumers. End users of the APIs interact exclusively with the Developer Portal and the documentation site; the monitoring layer is entirely invisible to them.

The stack is composed of three components with distinct responsibilities: Promtail collects raw log data at the source, Loki stores and indexes it, and Grafana queries and visualises it. None of these components sit in the request path, so their operation has no effect on API response times.

4.6.1 Log Collection with Promtail and Loki

WSO2 writes all gateway activity — inbound requests, authentication events, errors, and internal engine messages — to a file named `wso2carbon.log`. This file is stored in the `wso2_logs` named Docker volume, which is mounted simultaneously into the WSO2 container (for writing) and the Promtail container (for reading). This shared volume is the coupling point between the gateway and the monitoring stack, requiring no modification to WSO2 itself.

Promtail is configured via `promtail.yml` to tail this log file in real time. Because the WSO2 internal Synapse engine (`Synapse-PT-HttpComponents-NIO`) produces a large

volume of internal forwarding messages for every client request — effectively duplicating entries in the log — a pipeline stage was added to the Promtail configuration to discard these internal entries via a regular expression filter. Only log lines corresponding to actual client requests are forwarded to Loki, keeping the stored log volume lean and the queries fast.

Loki receives the filtered log stream and indexes it using label sets (e.g. job name, container name). It stores the raw log lines as compressed chunks and builds a sparse index over the labels, making it efficient for log aggregation without the overhead of full-text indexing. Queries are expressed in LogQL, Loki's query language, which allows filtering, parsing, and aggregating log data in the same way that PromQL is used for metric data.

4.6.2 Grafana Dashboard

Having structured, queryable access to gateway logs unlocks a range of operational insights that would be impossible to obtain from the APIs themselves. The Grafana instance is provisioned automatically from versioned configuration files — datasource definitions in YAML and dashboard layouts in JSON — meaning the entire monitoring interface is restored without manual setup whenever the stack is restarted.

Two dashboards are provided, each targeting a different operational concern.

The **APIs Dashboard** focuses on business-level traffic analysis. Because all requests pass through the WSO2 gateway and are logged uniformly, it is possible to compare the relative popularity of each API, identify which endpoints are called most frequently, and detect usage trends over time. Pie charts show the distribution of traffic across the API catalogue, making it immediately apparent which services drive the most load. Time-series panels reveal peak usage hours — for instance, the canteen menu API predictably spikes around midday — allowing the administrator to anticipate load and plan maintenance windows accordingly. In the following example, illustrated in Figure 4.6, one can see the service's logs from the last hour and see that the `Parques` API was not used, as well as compare the usage of the ones that were. .

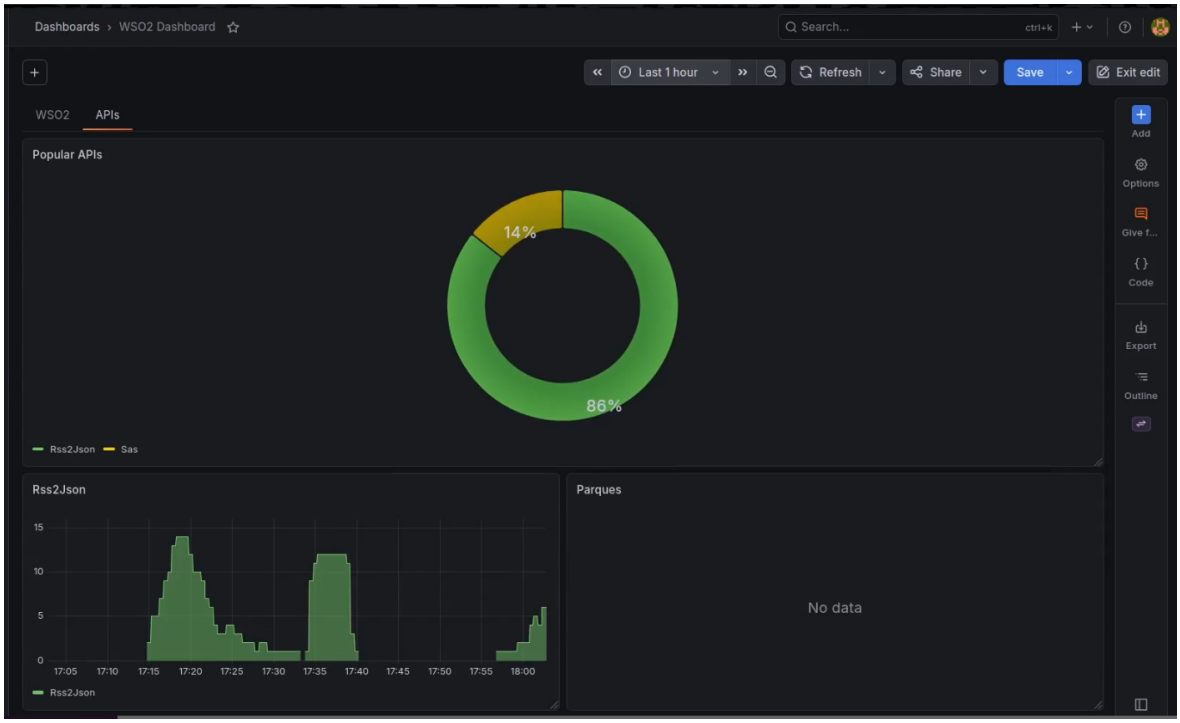

Figure 4.6: Grafana APIs Dashboard — traffic distribution and per-endpoint usage

The **WSO2 Dashboard** focuses on gateway health. It surfaces warning and error counts from the WSO2 logs, highlights `403 Forbidden` responses (indicating rejected or misconfigured access attempts), and flags critical exceptions such as `OutOfMemoryError` or `FATAL` entries. An alerting rule monitors for these critical patterns continuously; when triggered, it dispatches a notification to the administrator. The rule is calibrated with `noDataState: OK` so that periods of low or zero traffic — expected outside academic hours — do not generate false positive alerts. In the following example, illustrated in Figure 4.7, one can see the gateway's logs from the last hour.

Figure 4.7: Grafana WSO2 Dashboard — gateway health, error rates, and warnings

Together, the two dashboards give the administrator a complete picture of the platform: one lens focused on *who is using what*, and another on *whether the system is healthy*. This separation of concerns makes it straightforward to triage issues — a spike in 403 errors on the WSO2 dashboard points to an authentication or subscription problem, while an unexpected drop in traffic on the APIs dashboard may indicate a backend outage.

Chapter 5

Results

5.1 Expected Results

At the end of this project the expected result is to have a fully operational platform that operates in accordance with the objectives that were proposed. The system will be completely secured, operating with Keycloak as the identity provider, to prevent any unwanted access to the private APIs implemented. The platform should also be intuitive and easy to use by students, professors and external developers alike that aim to integrate any of the available APIs into their own projects. To support this, a comprehensive documentation website will be created, covering each endpoint and method available, alongside an interactive testing mechanism that allows users to try all those methods directly in the browser. Due to the modernization and different implementations, such as the usage of Memcached caching and other technologies, responses are expected to be processed in under 3 seconds under normal operating conditions. Finally, all the work done will remain compatible with systems already integrated with the previous platform, as the endpoints will stay the same, avoiding service disruption and ensuring continuity for existing consumers.

5.2 Achieved Results

Finished the project, all expected results were achieved. Even though the original plan of integrating with the university's infrastructure was not possible, the final result is a fully operational platform that meets all the objectives proposed at the beginning of the project. Keycloak is acting as a substitute for the university's identity provider, and all APIs are protected behind it, ensuring that only authenticated users can access them. The documentation portal is live and provides comprehensive information about each endpoint, along with an interactive testing interface that allows users to try out the APIs directly from their browsers. Since the endpoints are the same, the platform

is ready to be migrated if necessary.

5.2.1 APIs Delivered

The platform delivered four working APIs that cover the main use cases identified for the project. The **SAS** API exposes canteen menu information while preserving the legacy response formats used by existing consumers. The **Parques** API provides real-time parking occupancy data through a REST layer on top of the original SOAP service, making the information easier to integrate with modern clients. The **Rss2Json** API converts external RSS feeds into JSON or JSONP, offering a simple way to consume feed content programmatically. Finally, the **Access Points** mock API demonstrates how new institutional services can be added to the platform, providing a realistic example of the architecture's extensibility.

5.2.2 Security and Access Control

...

5.2.3 Performance(atualizar mais tarde)

The main performance target stated in the report was to keep API responses under 3 seconds under normal operating conditions. To support that objective, the platform was designed to minimise unnecessary upstream calls and to serve repeated requests from cache whenever possible. The SAS API stores the full week's menu data in Memcached and filters it in memory, while also using extttETag and extttCache-Control headers to avoid resending unchanged data. The Rss2Json API follows a stale-while-revalidate approach, returning cached content immediately and refreshing it in the background. The Parques API also reduces overhead by wrapping the original SOAP service behind a lightweight REST interface, so the most expensive part of the request path is isolated and only executed when fresh data is needed.

5.2.4 AI Acknoledgment

...

5.2.5 Known Limitations and Future Work

When we began this project, we wanted to make the system available to the public. That is something that we still would like to see happen in the future, as it would be a great way to validate the work done and to provide a useful service to the academic

community. Adding to that, we expect that the API catalogue will grow over time, providing more and more support for academic activities and keeping information services up to date for future modernization if necessary.

5.3 Projected Schedule

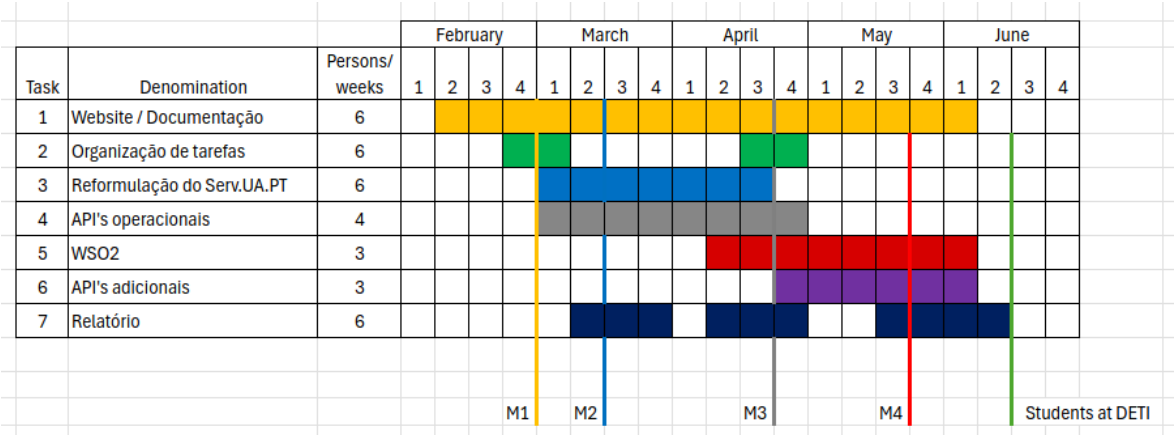

Figure 5.1: Projected Schedule

The planned work is organised into seven main tasks, distributed between February and June. Website and documentation activities span almost the entire period, providing continuous support to the remaining tasks. Initial weeks focus on task organisation and the refactoring of existing services, followed by the implementation and stabilisation of the core institutional APIs. In the final phase, additional APIs are integrated and the project report is completed, ensuring that the technical work is accompanied by up-to-date documentation and validation milestones.

Bibliography

[1] P. M. G. Ferreira. Open data @ ua. Dissertação de mestrado, Universidade de Aveiro, 2012.

[2] P. M. G. Ferreira and F. A. Gouveia. Academic playground & innovation. Relatório final, Universidade de Aveiro, April 2012.

[3] R. A. F. Jesus. Dinamização e divulgação da plataforma api - academic playground & innovation. Dissertação de mestrado, Universidade de Aveiro, 2014.